

TAREA - Reto: Calculadora de IMC Avanzada Multipantalla

App: Calculadora de Índice de Masa Corporal (IMC) con Validación y Navegación

Objetivo: Integrar el manejo de estados (`remember`), validación de entradas de usuario, y navegación entre pantallas enviando múltiples parámetros mediante Jetpack Compose.

Requisitos:

1. Composables y Layout

- Usar `Column` y `Row` para organizar la interfaz de manera limpia.
- Usar `Modifier` para agregar `padding`, definir tamaños y centrar el contenido.
- Usar `Spacer` para mantener márgenes adecuados entre los elementos.

2. Pantalla 1: Ingreso de Datos, Estados y Validación

- Implementar tres `TextField` con sus respectivos estados (`mutableStateOf`):
 - **Nombre** del usuario.
 - **Peso** (en kilogramos).
 - **Altura** (en metros).
- **[RETO 1] Validación:** El botón de "Calcular" solo debe ejecutar la navegación si los campos de peso y altura contienen valores numéricos válidos (mayores a cero). Si están vacíos o son inválidos, debe aparecer un `Text` de color rojo indicando "Por favor, ingresa valores válidos".
- Al presionar el botón de "Calcular" de forma exitosa:
 - Calcular el IMC (Fórmula: $\text{Peso} / (\text{Altura} * \text{Altura})$).
 - Navegar a la segunda pantalla enviando **dos parámetros**: el Nombre y el IMC calculado.

3. Navegación y Pantalla 2 (NavHost)

- Implementar el `NavHost` con `rememberNavController()`.
- **[RETO 2] Múltiples Parámetros:** Definir la ruta de la segunda pantalla para que acepte dos argumentos: `"resultado/{nombre}/{imc}"`.
- En la Pantalla 2, mostrar un saludo personalizado usando el parámetro recibido: `"Hola [Nombre], tu resultado es:"`.
- Mostrar el IMC calculado con un decimal.
- Utilizar una estructura `when` para clasificar el IMC:
 - Menor a 18.5 -> "Bajo peso"

- 18.5 a 24.9 -> "Peso normal"
- 25.0 a 29.9 -> "Sobrepeso"
- 30.0 o más -> "Obesidad"
- **[RETO 3] Interfaz Dinámica:** El color del `Text` que muestra la categoría debe cambiar dependiendo del resultado (ej. Verde para "Peso normal", Rojo para "Bajo peso" y "Obesidad", Naranja para "Sobrepeso").
- Un `Button` para "Volver" que utilice `popBackStack()` para regresar a la pantalla de captura de datos y limpiar los campos.

4. Git y GitHub

- Crear un repositorio público en GitHub para la tarea.
- **Mínimo 5 commits** que reflejen un progreso lógico (ej. "Pantalla 1 y UI", "Lógica de estado y validaciones", "Configuración de rutas", "Pantalla 2 terminada", "Retos visuales completados").

Entregables:

1. **Link del repositorio** en GitHub (asegúrate de que sea público).
2. **Screenshots (Capturas de pantalla)** de la app funcionando (Una captura de la pantalla 1 con datos llenos y otra de la pantalla 2 con el resultado colorido).
3. **Link de un video** (YouTube, Drive, etc.) donde expliques brevemente:
 - Cómo lograste pasar los dos parámetros en la ruta.
 - Cómo implementaste la validación en la primera pantalla.
 - Una prueba rápida de la app corriendo en el emulador.
4. *Nota: La app debe compilar y ejecutarse sin cerrarse forzosamente (crashear) bajo ninguna circunstancia.*